

DAY 4 IMPLEMENTATION

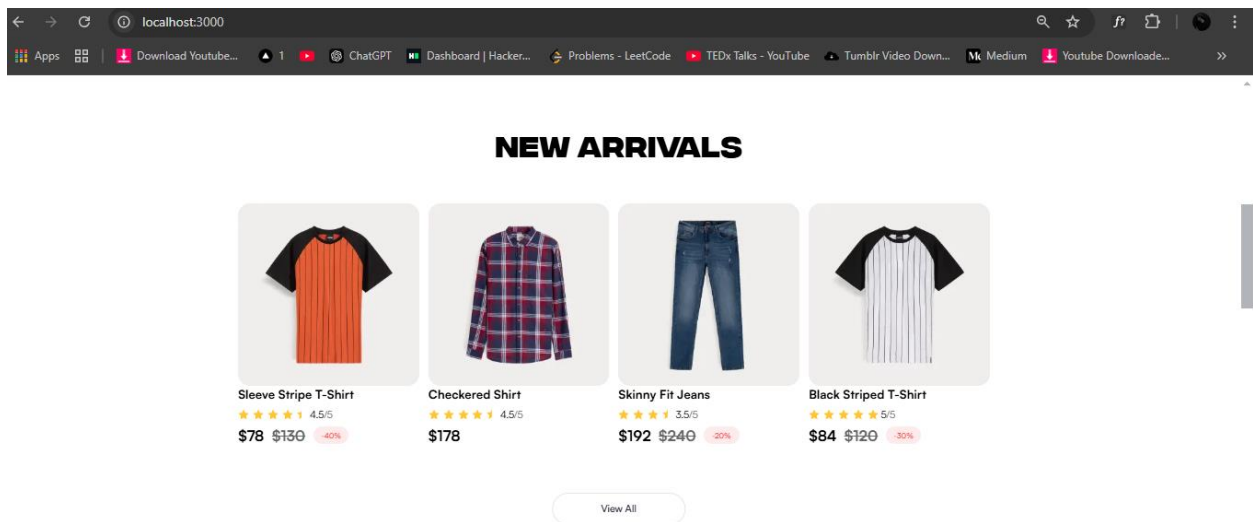
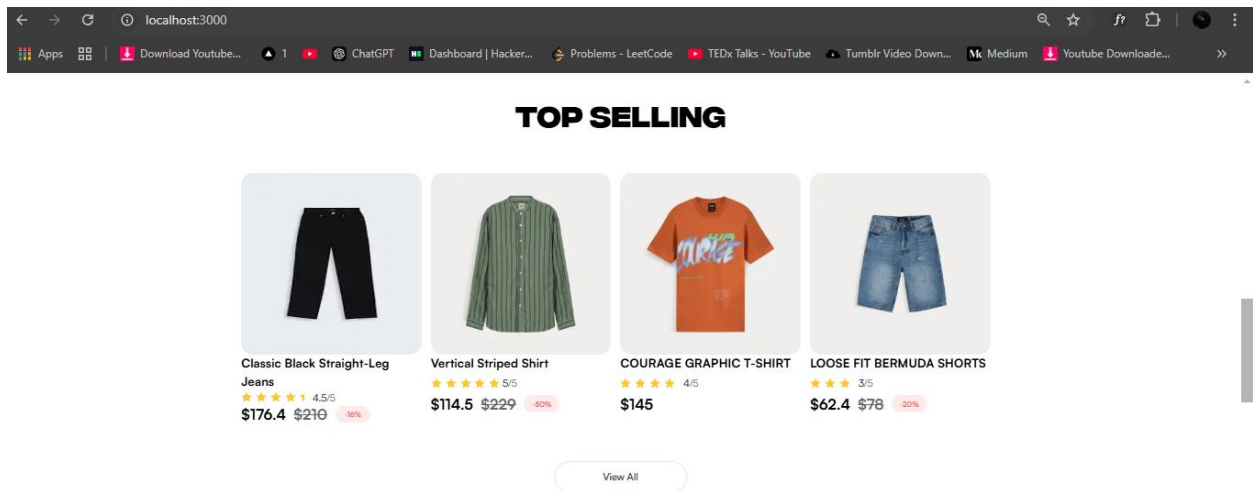
(Market Place: E-Commerce)

Project Overview

In this project, I created a dynamic e-commerce website using **Next.js**, **Sanity CMS**, and **Tailwind CSS**. The website includes a **Product Listing Page**, **Product Detail Page** (with dynamic routing), **Category Page** (with dynamic routing), and a **Cart Page**

Product Listing Component:

Description: In the product listing page, i displayed the products dynamically in a grid layout. Each product card includes essential details like the product name, price, image, rating and discount.



Product Listing Code Snippet:

```

<div className='w-[1240px] h-[413px] mx-auto mt-[70px] mb-[700px] flex flex-row justify-between'>
  {topsells.map((item:any, i:number) => {
    return (
      <div key={i}>
        <Link href={`/${item.category}/${item.slug}`}>
          <div className="w-[295px] h-[298px] z-10 rounded-[20px] bg-[#F0EEED] flex justify-center"
            <Image
              src={item.imageUrl}
              alt="shirt"
              width={296}
              height={444}
              className='hover:scale-125 duration-500'
            />
          </div>
        </Link>
        <div className="w-[295px] h-[98px] flex flex-col justify-between">
          <p className="font-Satoshi font-[700] text-[20px] text-black capitalize">
            {item.title}
          </p>

          <div className="max-w-[160px] h-[19px] flex flex-row gap-[13px] items-center">
            <div className="max-w-[104px] h-[18.49px] flex flex-row gap-[5.31px]">
              /**logic lagai hai jo rating kee base pay stars generate kray gee ratibg mtlb 2 3 4 !
              {Array.from({ length: Math.floor(item.rating) }).map(
                (_, i) => (
                  <div key={i}>

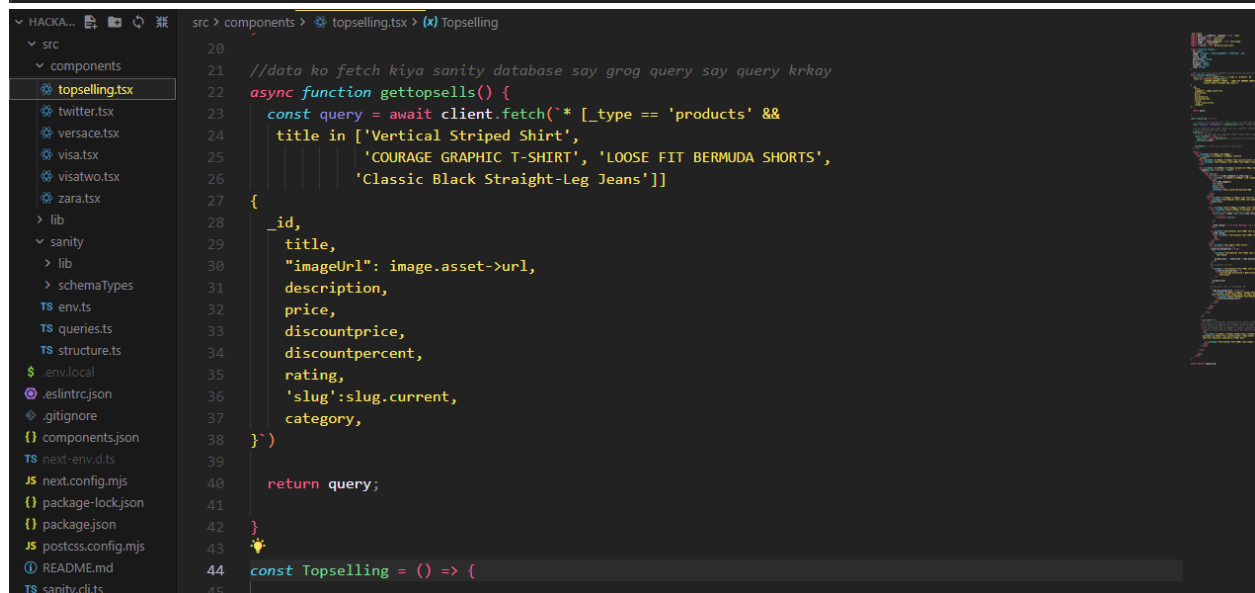
```

```
const Topselling = () => {

  // State to store fetched data <any[]> means kisi bhee type ka data store hoskta
  const [topsells, setTopsells] = useState<any[]>([]); //empty array shuru main ([])

  //ab ye component pehli baar render hota hai, useEffect chalega aur data
  //fetch karne ka kaam shuru karega
  useEffect(() => {
    // Fetch the data when the component mounts (mount means render)
    const fetchData = async () => {
      const data = await gettopsells(); //function ke through Sanity se data fetch hota hai.
      setTopsells(data);
    };

    fetchData(); // Call the function to fetch data
  }, []);
}
```



Product Detail Page (Dynamic Route) :

Description: The product detail page uses **dynamic routing** in Next.js and fetches detailed product data from **Sanity CMS** based on the product's SLUG. It displays product information such as description, price, and available sizes/colors dynamically.

Product Detail Code Snippet:

```
const Slug = ({ params }: { params: { slug: string } }) => {

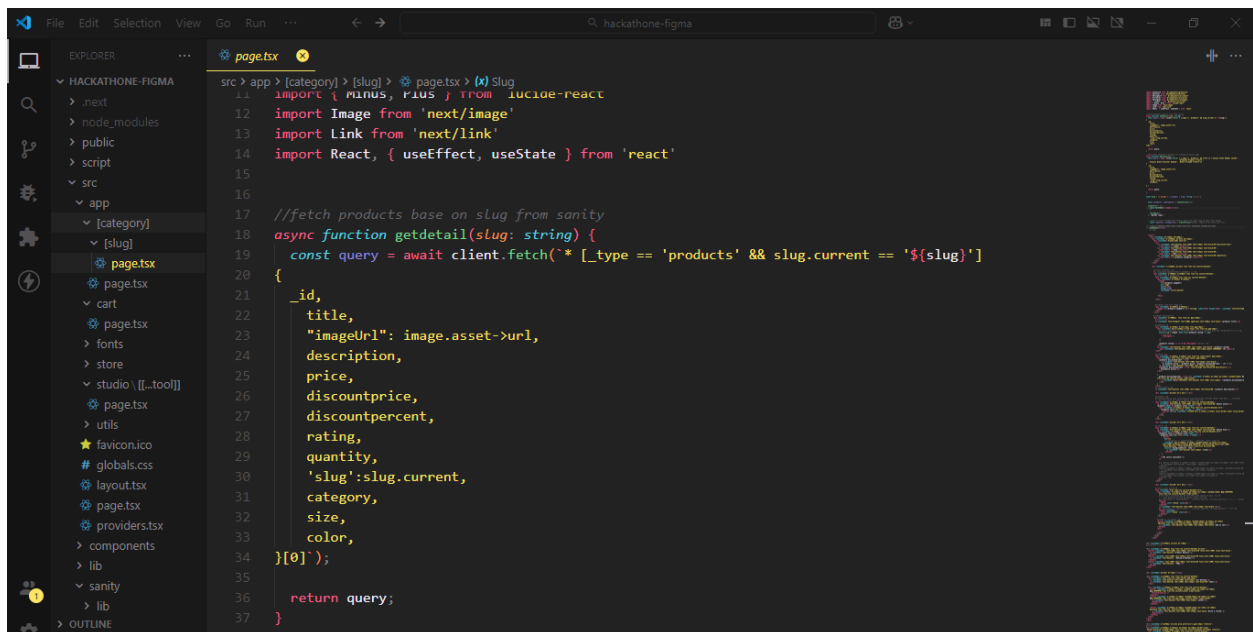
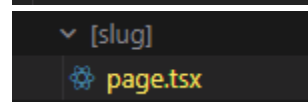
  const [products, setProducts] = useState<any>({})

  useEffect(() => {
    const fetchData = async () => { ...
    };

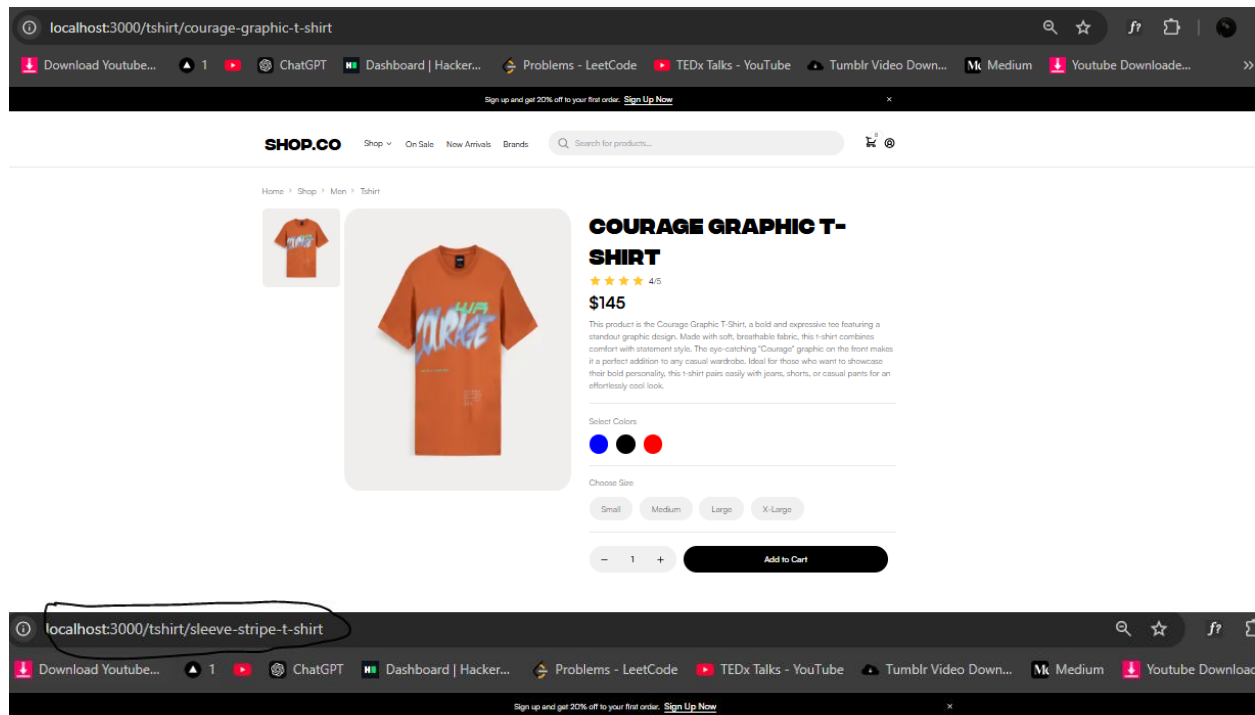
    fetchData();
  }, [params.slug]);

  // State to store fetched data <any[]> means kisi bhee type ka data store hoskta
  const [specific, setSpecific] = useState<any[]>([]); //empty array shuru main ([])

  //ab ye component pehli baar render hota hai, useEffect chalega aur data...
  useEffect(() => { ...
  }, []);
```



Product Detail UI View

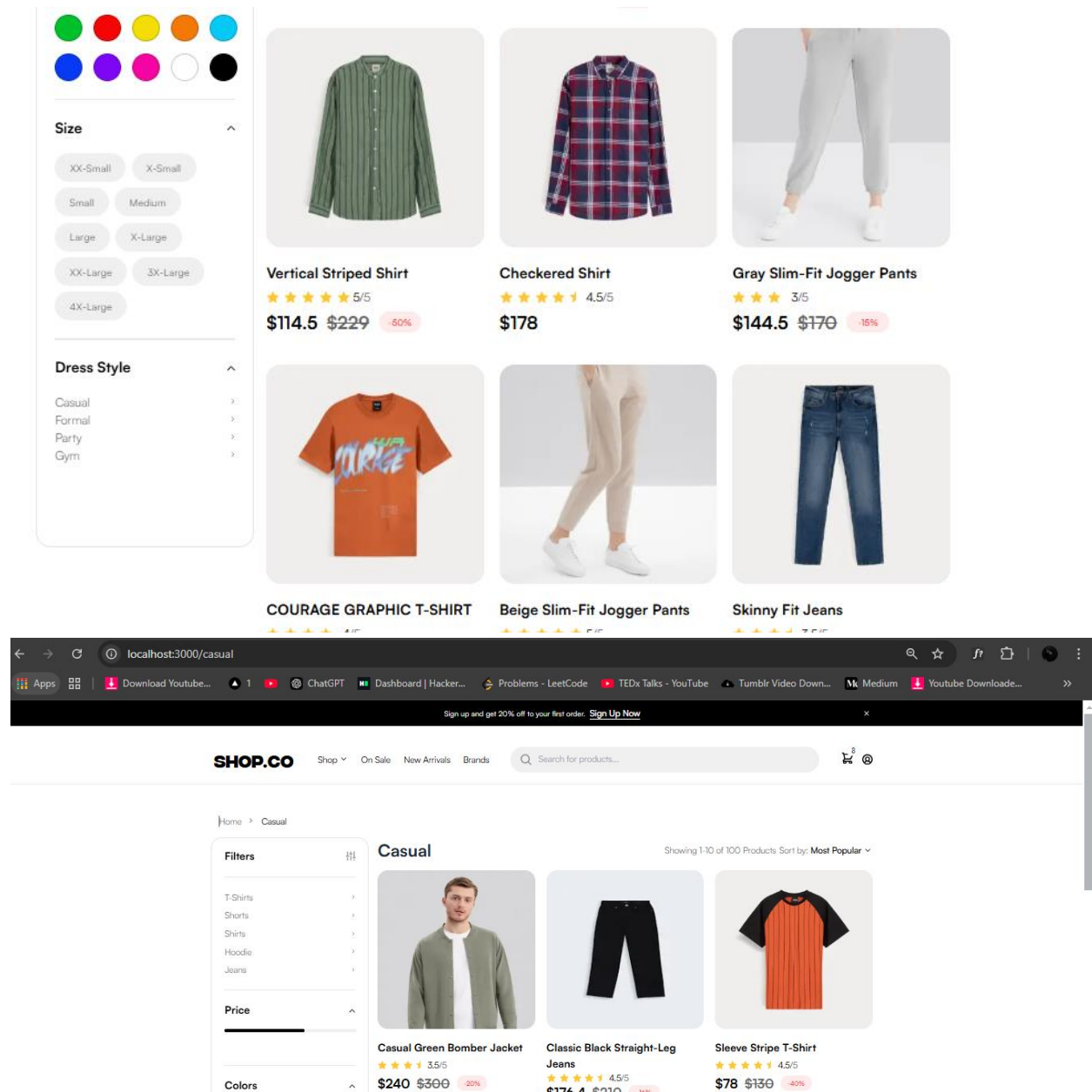


Category Page (Dynamic Route) :

Description The category page dynamically fetches categories from **Sanity CMS**, allowing users to filter products based on the selected

category. It provides an organized view of products belonging to each category ("shirts, shorts").

UI View Category Page:



Code Snippet Category Page:

```

{/**Products Card*/}
<div className='grid grid-cols-2 md:grid-cols-3 gap-x-[15px] md:gap-x-[20px] gap-y-[39px] mt-3'>
  {products.map((item:any, i:number) => {
    return (
      <div key={i}>
        <Link href={`/${item.category}/${item.slug}`}>
          <div className="w-[295px] h-[298px] rounded-[20px] bg-[#F0EEEE] flex justify-center over
            <Image
              src={item.imageUrl}
              alt="shirt"
              width={296}
              height={444}
            />
          </div>
        </Link>
        <div className="max-w-[295px] h-[98px] flex flex-col justify-between mt-[21px]">
          <p className="font-Satoshi font-[700] text-[20px] text-black capitalize">
            {item.title}
          </p>

```

```

page.tsx
src > app > [category] > page.tsx > (x) Category > fx getCategory > (x) query
16  const Category = ({params}: {params: {category: string}}) => {
150  const [products, setProducts] = useState<any[]>([]),
151
152  useEffect(() => {
153    // Fetch the data when the component mounts (mount means render)
154    const fetchData = async () => {
155      const data = await getCategory(); //function ke through Sanity se data fetch hota hai.
156      setProducts(data);
157    };
158
159    fetchData(); // Call the function to fetch data
160  }, []);
161
162
163  return (
164    <div className='max-w-[1240px] mx-auto my-[50px]'>
165      {/** <div className="divider w-[90%] mx-auto md:w-[1240px]"></div> */}
166      <BreadcrumbWithCustomSeparator/>
167      <div className='mt-[20px] flex'>
168        {/**Main div*/}
169        <div className='max-w-[295px] h-[1220px] rounded-[20px] py-[20px] px-[24px]
170          border-[1px] border-black/10 md:block hidden'>
171          {/**div jismain filter heading or filter icon hai */}
172          <div className='w-[247px] h-[27px] flex flex-row justify-between items-center'>
173            <p className='font-Satoshi font-[700] text-black text-[20px]'>Filters</p>
174            <Filter/>

```



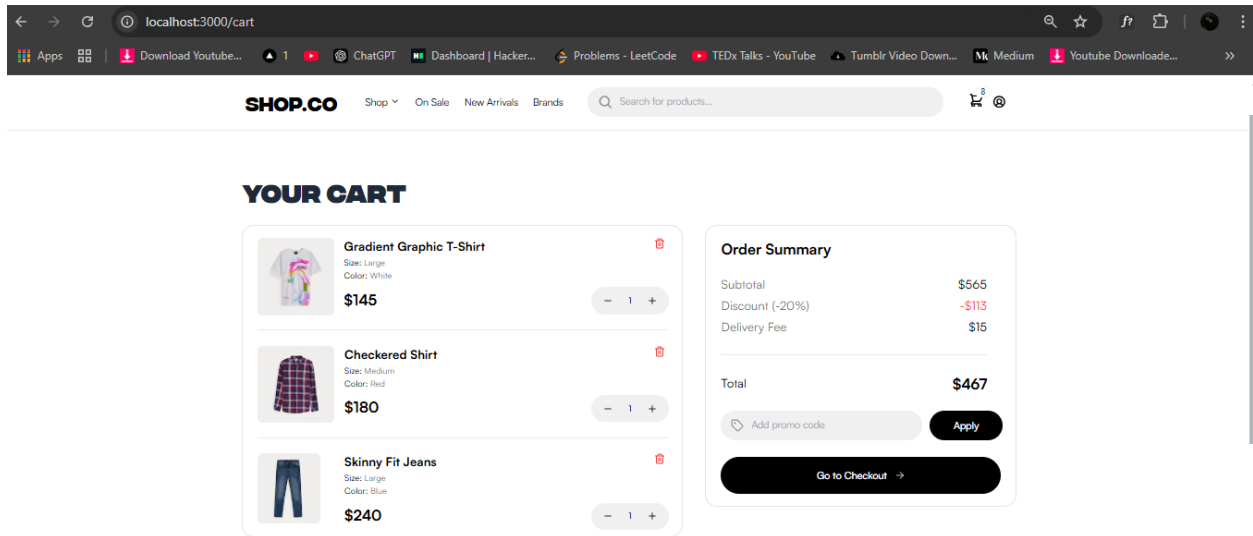
```

page.tsx
src > app > [category] > page.tsx > (x) Category > fx getcategory > (x) query
16  const Category = ({params}: {params: {category: string}}) => {
18  > //  const product: Product[] = [ ...
131
132  async function getcategory(){
133    const query = await client.fetch(`
134      * [_type == 'products'][0..8]
135      {
136        _id,
137        title,
138        "imageUrl": image.asset->url,
139        description,
140        price,
141        discountprice,
142        discountpercent,
143        rating,
144        'slug':slug.current,
145        category,
146      }`)
147    return query;
148  }
149
150  const[products, setProducts] = useState<any[]>([]);
151
152  useEffect(() => {
153    // Fetch the data when the component mounts (mount means render)
154    const fetchData = async () => {
155      const data = await getcategory(); //function ke through Sanity se data fetch hota hai.
156      setProducts(data);

```

Cart Page

Description: I have created a cart page where users can add items, remove items, and adjust the quantity of items (increasing or decreasing as needed). The order summary on right displays the total price, discount, and subtotal, providing a clear breakdown of the pricing details.



Steps Followed

1. Set up the Next.js Project:

- Created the project using create-next-app.
- Installed dependencies: **Sanity client, Tailwind CSS**, etc.

2. Sanity CMS Setup:

- Set up **Sanity CMS** for products. Created a product schema with fields like name, image, price, category, description etc.
- Migrated product data from an provided API to **Sanity CMS** made changes in api added fields on my project requirment.

3. Integrating Sanity Data:

- a. Used **Sanity client** to fetch data dynamically.
- b. Displayed data on the **Product Listing Page showing on landing page.**

Used dynamic routes (`[slug]page.tsx`) for the **Product Detail Page** and fetched product data.

4. Product Listing Page:

- a. Displayed products in a grid using **Sanity client**.

5. Dynamic Product Detail Page:

- a. Created dynamic routes and displayed detailed product information from **Sanity CMS**.

6. Dynamic Category Page:

- a. Created a category page that dynamically fetches and filters products based on selected categories.

7. Cart Page:

- a. Designed the layout for the cart page where user selected products will display.

Challenges Faced

- **Dynamic Routing and Data Fetching:** Faced challenges setting up dynamic routes. Solved by studying Next.js documentation.

- **Sanity CMS Integration:** Issues arose with setting up schemas and fetching data. Resolved by testing queries in **Sanity Studio** and adjusting parameters.
- **Tailwind CSS Layout:** Initially struggled with responsive grid layouts.
- **Cart Functionality:** Used Redux-Toolkit for cart state management and redux persist to persist cart state means your cart item will always remain in cart even if u refresh the page whenever u refresh the page state data will come from localstorage.

Error Solving Approach

1. **Sanity Fetching Errors:** Debugged by testing queries in **Sanity Studio** and verifying them with **Sanity client**.
2. **Dynamic Route Errors:** Fixed by ensuring the correct product slug was passed in the URL.

Conclusion

This project taught me how to integrate **Sanity CMS** with **Next.js** and how to build dynamic pages.